E-Parada: A Digital Marketplace for Parking Spaces

Capstone Defense Presentation Outline (Copy-Paste Ready) — College of Engineering and Architecture, NU Laguna

Presenters: Bustria, Christian Jake C. • Gamad, Leticia C. • Ovilla, Shan Jenis C. • Valiña, Resha Lorraine P.

✓ E-Parada (15) Structure + E-Parada (16) Tech Stack ✓ IT Developer Evaluation Only ✓ 2D Floor Plan Slot Mapping ✓ Supabase PostgreSQL 15+ ✓ Zero Business Model / Commission Fees

CHAPTER 1: INTRODUCTION (Objectives, Scope & Limitations, Conceptual Framework)

SLIDE 1 Objectives of the Study

Chapter 1

General Objective

- To enhance efficiency, effectiveness, and reliability in parking management by designing and developing **E-Parada**, a software-driven, cross-platform digital marketplace that connects residential and commercial parking space providers with motorists seeking available parking spaces in Calamba City, Laguna.

Specific Objectives

1. **To design** a centralized parking marketplace architecture that incorporates parking area and 2D slot management, OCR-assisted vehicle registration, location-based searching, cryptographic QR code access workflows, automated duration billing computation, and reservation-bound communication.
2. **To develop** the E-Parada software ecosystem—comprising the **Flutter Android mobile application** and the **Laravel 12 administrative web portal** backed by **Supabase (PostgreSQL 15+)**—utilizing the **Incremental Development Model** and integrating third-party APIs for mapping (Google Maps), OCR (Tesseract), voice calling (WebRTC), push notifications (FCM), and payment processing (GCash via PayMongo).
3. **To test and evaluate** the software quality of the developed system through **IT developer evaluation** and objective technical benchmarking using the **ISO/IEC 25010:2011 Software Product Quality Model** in terms of:
    - **Functional Suitability • Performance Efficiency • Usability • Reliability**

> 🎙 **Presenter Note:** "Our study is anchored on three specific objectives: designing a 100% software-driven parking marketplace architecture, developing the cross-platform Flutter and Laravel ecosystem using the Incremental Model, and evaluating software quality with IT developers under ISO/IEC 25010:2011."

SLIDE 2 Scope of the Study

- **Cross-Platform Access:** Android Mobile Application (Flutter) & Administrative Web Portal (Laravel 12) synchronized via **Supabase (PostgreSQL 15+)**.
- **Target Coverage:** Residential driveways, vacant lots, commercial establishments, and select light-industrial parking spaces in **Calamba City, Laguna**.

Stakeholder Role	Key Functional Scope
Parking Space Providers	• Account & property ownership verification • Parking space listing with images, operating schedules, & overstay grace rules • Interactive **2D floor plan slot configuration** (dimensions, clearance, supported vehicle types) • Hourly rate setting within administrator-approved fee boundaries • Reservation approval/rejection & **QR code entry/exit scanning** (with manual `RES-#` fallback) • Real-time active session monitoring & cumulative earnings tracking
Drivers / Motorists	• Account registration & valid ID upload • **OCR-assisted vehicle registration** (Tesseract OCR license plate extraction) • Geolocation search, vehicle dimension filtering, & **Google Maps** turn-by-turn navigation • Slot reservation & **HMAC-SHA256 QR pass** generation • Automated duration & overstay billing with **Cash or GCash (PayMongo)** settlement • Reservation-bound **in-app chat & WebRTC audio calling** + incident reporting
System Administrators	• KYC verification of users, property documents, and OCR-extracted vehicles • Listing approval & minimum/maximum parking fee range governance • Real-time reservation, QR log, and payment monitoring • Dispute mediation & **CSV forensic audit report generation**

SLIDE 3 Limitations of the Study

1. **Platform Support:** Limited to Android devices (Android 8.0+) and standard web browsers.
2. **Connectivity:** Requires a stable internet connection for real-time database synchronization, mapping, and payment webhooks.
3. **Geolocation Precision:** GPS accuracy depends on mobile device location hardware and the Google Maps Platform API.
4. **Gate Validation:** Entry and exit QR validation is performed through the parking space provider's authorized mobile device.
5. **Hardware Independence (Exclusions):** Operating strictly as a **software-driven solution**, the system **does not include**:
   - Automated physical gate barriers or turnstiles • RFID tags/readers • Stationary IoT ultrasonic/IR ground sensors • AI parking prediction • Live CCTV streaming
6. **Verification Scope:** Relies on administrative document review within the platform (no automated lookup against external LTO or PNP government databases).
7. **Billing & Rate Boundaries:** Automated billing is strictly computed from validated QR `time-in` and `time-out` scans; provider rates must remain within admin-established fee ranges.
8. **Third-Party Gateway Dependency:** Cashless transactions rely on the external GCash/PayMongo payment gateway.
9. **Evaluation Scope:** System evaluation is conducted through controlled technical benchmarking and end-to-end software quality assessment by **IT developers** simulating Calamba City parking scenarios, rather than large-scale public deployment.

SLIDE 4 Conceptual Framework (IPO Model)

Figure 1

INPUT	PROCESS	OUTPUT
1. System Requirements • Driver, Provider, & Admin functional requirements • Android 8.0+ & Web compatibility, GPS & internet **2. System Design Inputs** • Use Case, System Flowchart, DFDs, & Crow's Foot ERDs • 2D Floor Plan UI/UX wireframes, REST API & WebRTC specs **3. Development Data Inputs** • User IDs, vehicle specs & plate photos, space dimensions, 2D slot coordinates, rate bounds, & QR logs **4. Development Tools (Updated E-Parada 16 Stack)** • **Mobile:** Flutter 3.x / Dart SDK • **Backend:** PHP 8.2 / Laravel 12 (Docker & Nginx on Render) • **Database:** Supabase (Managed PostgreSQL 15+ & Realtime) • **APIs & Libraries:** Google Maps API, Tesseract OCR CLI, Google ML Kit & CameraX (`mobile_scanner`), `qr_flutter`, WebRTC (`flutter_webrtc`), FCM, Brevo SMTP, & PayMongo API v1 (GCash) **5. Testing & Evaluation Inputs** • 20 Functional & Security Test Cases • Technical Benchmark Workloads • ISO/IEC 25010:2011 Evaluation Instrument for IT Developers	**1. System Planning & Analysis** • Analysis of Calamba City parking constraints • Formulation of objectives, scope, & role requirements **2. System Design** • UI/UX prototyping & 2D slot layout design • PostgreSQL schema modeling & REST API routing • HMAC-SHA256 QR & billing algorithm design **3. Incremental System Development** • **Increment 1:** Auth, RBAC, & KYC/Vehicle OCR verification • **Increment 2:** Space listing, rate bounds, & 2D floor plan slot mapping • **Increment 3:** Geolocation search, Google Maps routing, & concurrency-locked booking • **Increment 4:** QR entry/exit validation, duration/overstay billing, & GCash/PayMongo checkout • **Increment 5:** In-app chat, WebRTC calling, FCM/Brevo alerts, & Admin CSV reports **4. System Integration & Testing** • Alpha testing, regression testing, & end-to-end scenario validation **5. System Evaluation** • Technical benchmarking & **IT Developer evaluation** under ISO/IEC 25010:2011 (Weighted Mean & SD)	**E-Parada: A Digital Marketplace for Parking Spaces** • Verified Cross-Platform **Flutter Android Mobile App** & **Laravel Administrative Web Portal**

CHAPTER 2: REVIEW OF RELATED LITERATURE AND STUDIES

SLIDE 5 Literature Matrix

Chapter 2

[INSERT LITERATURE MATRIX TABLE HERE]

(Intentionally left blank as requested — paste your Literature Matrix table on this slide)

SLIDE 6 Literature & Design Synthesis

Chapter 2

- **1. Geolocation & Urban Navigation** *(Raza et al., 2022; Sartayeva et al., 2023; Jagelčák et al., 2024; Bastani et al., 2021; Grepon et al., 2023; Kim et al., 2023)*
  Consumer GNSS sensors experience multipath drift in dense urban corridors; coupling device GPS with the **Google Maps Platform API** provides reliable pin discovery and turn-by-turn routing to decentralized residential parking slots.
- **2. Smart Parking & Reservation Systems** *(Barde et al., 2022; Sudhakar et al., 2023; Gamage, 2025; Guzman et al., 2024; Larraquel, 2023)*
  Existing smart parking systems depend heavily on costly stationary IoT hardware (ultrasonic sensors, Raspberry Pi controllers, automated barriers) inside single facilities. E-Parada bridges this gap through a **100% software-driven architecture** using consumer smartphones and interactive **2D floor plan slot mapping**.
- **3. Digital Sharing Economy & Resource Optimization** *(Pouri & Hilty, 2021; Wang, 2025; Ahmed et al., 2023)*
  Digital resource-sharing platforms unlock underutilized private parking spaces (driveways and vacant lots) without requiring new physical infrastructure construction.
- **4. Cryptographic QR Validation & Replay Protection** *(Skurowski et al., 2022; Alsuhibany, 2025; Desale et al., 2025; Bekavac et al., 2024)*
  Static QR codes are vulnerable to screenshot sharing and forgery. E-Parada treats the QR code as a signed access token verified server-side via **HMAC-SHA256**, provider-space authority checks, and single-use state transitions.
- **5. Cross-Platform Synchronization & Transactional Integrity** *(Ali & Dauwed, 2022; Wahyudi et al., 2021; Kueh Wee Xin, 2025; Ahmed et al., 2024)*
  Combining **Flutter + Laravel 12 + Supabase (PostgreSQL 15+)** delivers strict ACID compliance and pessimistic row-level locking (`SELECT ... FOR UPDATE`) to prevent double-bookings, paired with WebSocket Change Data Capture (CDC) for real-time updates.
- **6. Automated Duration Billing & Cashless Settlement** *(Vovveti, 2024; Afable, 2024; Fabro et al., 2025)*
  Automated timestamp-delta billing eliminates manual calculation errors, while **GCash (via PayMongo API v1)** provides trusted, webhook-verified cashless settlement alongside cash options.

CHAPTER 3: METHODOLOGY

SLIDE 7 Research Design and Development Method

Figure 26

Research Design: Descriptive-Developmental Method

- **Developmental Aspect:** Systematic software engineering, multi-tenant PostgreSQL database modeling, RESTful API integration, and cross-platform implementation of the Android mobile application and administrative web portal.
- **Descriptive Aspect:** Quantitative assessment of software product quality by **IT developers** using the **ISO/IEC 25010:2011** standard and objective technical performance benchmarks.

Software Development Model: Incremental Development Model (5 Sequential Increments)

Development Phase / Increment	Core Engineering Deliverables
Phase 1: Planning & Design	Calamba City domain analysis, UI/UX wireframes, Crow's Foot ERDs, REST API specs, & Docker/Supabase setup.
Increment 1: Core User Access	User registration, Laravel Sanctum RBAC, KYC ID upload, & **Tesseract OCR** license plate extraction.
Increment 2: Parking & 2D Slots	Parking space listing, admin fee boundary enforcement, & **interactive 2D floor plan slot mapping**.
Increment 3: Search & Booking	Geolocation proximity filtering, **Google Maps** routing, & pessimistic row-locked booking (`SELECT ... FOR UPDATE`).
Increment 4: QR & Billing	**HMAC-SHA256 QR** entry/exit scanning (Google ML Kit/CameraX), duration/overstay fee math, & **GCash (PayMongo v1)** + Cash settlement.
Increment 5: Comms & Admin	Reservation-bound chat, **WebRTC** voice calling, **FCM & Brevo** notifications, dispute mediation, & Admin CSV audit reports.
Phases 3–5: Testing & Evaluation	Alpha/Regression testing, objective benchmarking, & **IT Developer evaluation** under ISO/IEC 25010:2011.

SLIDE 8 System Architecture, Use Case, & Database Schema

Figures 6, 7, & 9–15

Three-Column System Architecture (Figure 6) & Use Case Diagram (Figure 7)

- **1. Data Gathering Component:** Captures operational inputs from **Drivers** (search, vehicle specs, booking, payment) and **Parking Space Providers** (space listings, 2D slot layouts, rate bounds, QR validation) overseen by **System Administrators**.
- **2. Acquired Knowledge & System Features:** Synthesizes 18 core modules including RBAC, OCR plate verification, 2D slot compatibility, pessimistic concurrency control, HMAC-SHA256 QR validation, automated billing, and WebRTC/Chat coordination.
- **3. Computer Program Component (Updated Stack):**
  - **Client Tier:** Flutter 3.x / Dart (Android Mobile App & PWA) + Laravel Blade, Tailwind CSS 3.x, & Alpine.js 3.x (Web Portal).
  - **Application & API Tier:** PHP 8.2 / Laravel 12 RESTful API containerized with **Docker & Nginx on Render Cloud**.
  - **Data Tier: Supabase (Managed PostgreSQL 15+)** with Connection Pooler, Row-Level Security (RLS), and Realtime WebSockets.

Modular Relational Database Architecture (Figures 13–15)

- **Core Booking Database (Figure 13 — 6 Tables):** `users`, `vehicles`, `parking_spaces`, `parking_slots`, `reservations`, `parking_space_feedback`
- **Communications & Notification Database (Figure 14 — 5 Tables):** `reservation_messages`, `reservation_calls`, `reservation_call_signals`, `notifications`, `user_device_tokens` (FCM)
- **Security Audit & Telemetry Database (Figure 15 — 3 Tables):** `activity_logs`, `qr_transaction_logs`, `support_requests`

SLIDE 9 Proposed Prototype (Mobile & Web Interfaces)

Figures 19–23

Interface Module	Key Screens & Functional Highlights
a. Driver Mobile UI (Figure 19)	• **Browse Parking:** Google Maps interactive pin discovery & proximity/vehicle-compatibility filtering around Calamba City. • **Space Details & 2D Slot Map:** Operating schedules, hourly rates, and interactive **2D floor plan slot selection**. • **My Reservations:** 6-stage real-time status bar (*Requested → Approved → Checked In → Checked Out → Payment Sent → Paid*), encrypted **Entry QR Code**, reschedule/cancel controls, and provider chat/call. • **Driver Profile:** KYC verification badges & OCR-assisted vehicle registration slots (up to 3 vehicles).
b. Parking Owner Mobile UI (Figure 20)	• **Owner Dashboard:** Live counters for registered spaces, approved spaces, pending requests, and payments to verify. • **My Parking Spaces:** Property editor, 2D slot layout configurator, and operating schedule manager. • **Entry & Exit Checkpoint Validator:** Hardware-accelerated camera QR scanner (`mobile_scanner`) with manual `RES-#` fallback and checkpoint mismatch detection.
c. System Admin Mobile & Web UI (Figures 21 & 22)	• **Admin Reviews (KYC & OCR Audit):** Side-by-side verification of user IDs, property proofs, and OCR-extracted license plates vs. vehicle photos. • **Reservation & Platform Operations Monitor:** Live transaction tracking, dispute resolution, QR scan telemetry logs, support desk, and 1-click **CSV audit report export**.

SLIDE 10 Hardware and Software Components

Tables 1 & 2

Table 1: Hardware Specification

Category	Component	Specification	Description
Development Device	Laptop / Desktop PC	Intel Core i3 / AMD Ryzen 3, 8 GB RAM, 256 GB SSD	Used for coding, compiling Flutter builds, and managing backend/DB.
Mobile Test Device	Android Smartphone	Android 8.0+, 4 GB RAM, Camera, GPS, Microphone	Used for testing QR scanning, OCR plate capture, GPS, & WebRTC calls.
Network Connectivity	Wi-Fi Router / Hotspot	Stable broadband / 4G LTE connection	Provides internet connectivity between mobile client and cloud backend.

Table 2: Updated Software Specification (E-Parada 16 Stack)

Category	Component	Specification	Architectural Role
Mobile Framework	Flutter & Dart SDK	Flutter 3.x / Dart `^3.12.2`	Powers cross-platform Android mobile client & PWA.
Backend Framework	PHP & Laravel	PHP 8.2 / Laravel 12.0	RESTful API engine, Sanctum RBAC, HMAC signing, & billing math.
Database (DBMS)	Supabase (PostgreSQL)	Managed PostgreSQL 15+	ACID transactions, `SELECT ... FOR UPDATE` locking, RLS, & Realtime CDC.
Cloud Hosting	Docker & Render Cloud	Alpine Linux + Nginx + PHP-FPM	Multi-stage containerized production deployment with SSL.
2D Spatial Layout	2D Floor Plan Slot Engine	Custom Flutter Painter & HTML5 Grid	Renders interactive 2D parking lot lanes and angled slot layouts.
Web Frontend	Blade + Tailwind + Alpine	Tailwind 3.x / Alpine 3.x	Responsive administrative web portal and dashboard interface.
Mapping & OCR	Google Maps API & Tesseract	Maps SDK / Tesseract PSM 11	Geolocation routing & automated LTO license plate character extraction.
QR Scanner & Comms	ML Kit, CameraX, WebRTC, FCM	`mobile_scanner: ^7.4.0` / `flutter_webrtc`	Hardware-accelerated QR scanning, VoIP audio calling, & push/Brevo alerts.

Category	Component	Specification	Architectural Role
Payment Gateway	GCash via PayMongo API	PayMongo API v1	Authenticated cashless checkout sessions & automated payment webhooks.

SLIDE 11 Data Flow and Control Logic (Activity Diagram, DFDs, & Subroutines)

Figures 8, 16–18, 24, & 25

1. Activity Diagram (Figure 8) & 12-Phase Cross-Functional System Flowchart (Figure 24)

- Coordinates interactions across **Driver**, **Web & Mobile Client**, **Laravel Backend & Services**, **Parking Space Provider**, **System Administrator**, and **Supabase Database** swimlanes:
  - **Phases 1–3 (Onboarding & Auth):** Registration → Brevo email verification → Admin KYC/OCR approval → Role-based dashboard routing.
  - **Phases 4–6 (Discovery, Listing, & Booking):** Provider listing creation → Driver proximity/dimension search → Pessimistic slot lock → Provider approval → HMAC-SHA256 QR pass issuance.
  - **Phases 7–9 (Checkpoint Scan & Billing):** Gate Entry QR scan (`time_in`) → Active session monitoring → Gate Exit QR scan (`time_out`) → Automated duration + overstay billing → Cash or GCash (PayMongo) settlement → Provider earnings record updated → Driver feedback.
  - **Phases 10–12 (Governance & Audit):** Admin listing/user reviews, dispute resolution, feedback moderation, & CSV audit export.

2. Partitioned Data Flow Diagrams (Figures 16–18)

- **Level-0 DFD (Figure 16):** 6 Core Processes (*1.0 Auth, 2.0 Listings, 3.0 Search & Booking, 4.0 Check-In/Out, 5.0 Billing & Settlement, 6.0 Feedback & Audit*) across 6 Data Stores (`D1–D6`).
- **Level-1 DFD (Figure 17):** Decomposes **Process 3.0** into 6 subprocesses (*3.1 Query → 3.2 Compatibility Filter → 3.3 Validate Request → 3.4 Conflict Check → 3.5 Generate HMAC QR → 3.6 Dispatch Pass*).
- **Level-2 DFD (Figure 18):** Decomposes **Process 3.5** (*3.5.1 Extract Tuple → 3.5.2 Retrieve Secret Key → 3.5.3 Compute HMAC-SHA256 Hash → 3.5.4 Encode 2D Matrix → 3.5.5 Archive Token*).

3. Four Encapsulated Technical Subroutines (Figure 25)

- **Subroutine 1 (Tesseract OCR):** Grayscale + contrast boost + adaptive thresholding → PSM 11 extraction → LTO plate regex filter.
- **Subroutine 2 (Pessimistic Concurrency Locking):** `BEGIN` → `SELECT ... FOR UPDATE` on `parking_slots` → overlap check → `COMMIT` or `ROLLBACK`.
- **Subroutine 3 (Cryptographic QR Replay Guard):** Base64 decode → recompute `HMAC-SHA256` → verify single-use state → log to `qr_transaction_logs`.
- **Subroutine 4 (Automated Duration & Overstay Billing):** Compute `time_out - time_in` minus 15-min grace period → apply ceiling-rounded hourly & overstay rates → Cash or PayMongo GCash webhook verification.

SLIDE 12 Testing Environment, Phases, and Test Case Coverage

Tables 3 & 4

- **Testing Environment:** Physical Android smartphones (Android 8.0–14.0) + Android Studio Emulator + Desktop/Mobile Web Browsers connected to containerized PHP 8.2 / Laravel 12 & **Supabase PostgreSQL 15+** across Wi-Fi, 4G/LTE, and throttled low-bandwidth networks.

Table 3: System Testing Phases and Acceptance Criteria

Testing Phase	Scope and Methodology	Pass/Fail Acceptance Criteria
1. Alpha Testing (Unit & Modular)	Internal modular testing by the development team across all 5 increments (Auth, Listings, Maps, QR, Billing, GCash, WebRTC).	• 100% pass rate on core auth, booking, and billing unit test suites. • Zero critical DB integrity violations or unhandled exceptions.
2. System & Scenario Evaluation (End-to-End Validation)	End-to-end operational and technical evaluation conducted by purposively selected **IT Developers** executing complete Driver, Provider, and Admin workflows.	• ≥ 95% successful completion rate across end-to-end role journeys. • Zero occurrences of unresolved double bookings or unrecorded payments.
3. Regression Testing	Automated and manual re-testing of previously verified modules after bug fixes or increment integrations.	• 100% pass on all 20 test cases; zero regression bugs in reservation states and billing math.

20-Item Test Case Coverage Matrix Summary (Table 4)

- **TC-01 to TC-02 (Auth & RBAC):** Sanctum token issuance & `HTTP 403` unauthorized route blocking.
- **TC-03 to TC-04 (Vehicle & OCR):** Tesseract OCR extraction (≥ 90% precision) & duplicate plate rejection.
- **TC-05 to TC-06 (Listing & 2D Layout):** Admin fee boundary enforcement & responsive **2D floor plan slot grid** rendering.
- **TC-07 to TC-08 (Search & Maps):** Proximity/dimension filtering & Google Maps route calculation.
- **TC-09 to TC-10 (Booking & Concurrency):** Standard booking & **simultaneous race-condition stress test (`SELECT ... FOR UPDATE`)**.
- **TC-11 to TC-14 (QR Security):** Valid entry/exit logging, **replay attack rejection**, unauthorized scanner blocking, & manual `RES-#` fallback.
- **TC-15 to TC-20 (Billing, Comms, & Network):** 15-min grace period & overstay math, Cash confirmation, **GCash PayMongo webhook**, reservation-bound chat, **WebRTC call (< 250 ms)**, & network drop recovery.

SLIDE 13 Evaluation and Statistical Treatment

Tables 5 & 6

- **Evaluators & Sampling Technique: Purposive Sampling** of qualified **IT Developers and Software Engineering Professionals** who execute full Driver, Provider, and Admin workflows and evaluate the system under the **ISO/IEC 25010:2011 Software Product Quality Model**:
  - **A. Functional Suitability** (Items 1–3: *Completeness, Correctness, Appropriateness*)
  - **B. Performance Efficiency** (Items 4–6: *Time Behavior, Resource Utilization, Capacity*)
  - **C. Usability** (Items 7–12: *Recognizability, Learnability, Operability, User Error Protection, UI Aesthetics, Accessibility*)
  - **D. Reliability** (Items 13–16: *Maturity, Availability, Fault Tolerance, Recoverability*)

Table 5: Objective Technical Performance Benchmarks

Performance Metric	Evaluation Target Threshold	Measurement Instrument	Evaluated Component
REST API Latency	Mean Response < 350 ms (at 50 req/s)	Apache JMeter / Postman Runner	Laravel 12 API & Supabase Pooler
QR Validation Speed	Total latency < 600 ms	Physical Android Benchmark Timer	Base64 Decode & HMAC-SHA256
Concurrency Resilience	0 double-bookings (25 parallel reqs)	Python Concurrency Stress Script	PostgreSQL Row Lock (`FOR UPDATE`)
OCR Extraction Latency	Execution < 1.8 s; Precision ≥ 90%	50-Plate Automated Test Harness	Tesseract OCR Engine (PSM 11)
WebRTC Audio Latency	Packet latency < 200 ms	Chrome WebRTC Internals Logger	Flutter WebRTC P2P / Relayed Session
Payment Webhook Latency	Round-trip < 1.2 s	PayMongo Webhook Logger	PayMongo API v1 Event Listener

Statistical Treatment Formulas & Table 6: Likert Scale Interpretation Matrix

1. Frequency & Percentage
(Evaluator Technical Profile)
$$P = (f / N) \times 100$$

2. Weighted Mean
(Central Quality Rating)
$$\bar{X} = (\sum fx) / N$$

3. Standard Deviation
(Evaluator Rating Consistency)
$$SD = \sqrt{[\sum (X - \bar{X})^2 / (N - 1)]}$$

Scale	Weighted Mean Range	ISO/IEC 25010:2011 Quality Rating	Qualitative Level
5	4.21 – 5.00	Excellent / Highly Compliant	Outstanding software quality
4	3.41 – 4.20	Very Good / Compliant	High software quality
3	2.61 – 3.40	Fair / Moderately Compliant	Moderate software quality
2	1.81 – 2.60	Poor / Low Compliance	Substandard software quality
1	1.00 – 1.80	Very Poor / Non-Compliant	Unacceptable software quality